TECHNICAL SPECIFICATION & ALGORITHM WHITEPAPER

Lurelock: AI Training Mechanics & Mathematical Risk Formulation

An In-Depth Mathematical and Architectural Breakdown of Phishing URL Detection Using Extreme Gradient Boosting (XGBoost)

FRAMEWORK	FEATURE SPACE	ENSEMBLE SIZE	TEST ROC-AUC	TEST ACCURACY
XGBoost 3.4 + Scikit-Learn	**1,018 Dims (Hybrid Union)**	**300 Trees (Depth 6)**	**0.9996 (40,722 URLs)**	**99.87%**

1. End-to-End System Pipeline

Phishing URLs deceive users by imitating familiar brands (typosquatting), injecting nested authentication tokens, embedding bare IP hosts, or generating high-entropy domain strings. **Lurelock** detects these tactics without relying on static blacklists. The system runs locally and follows a 5-stage pipeline:



2. Dataset Engineering & Training Pipeline

The training engine in `src/dataset_cleaner.py` and `src/trainer.py` automates the ingestion, sanitization, and stratified partitioning of 203,610 ground-truth URLs:

- **Automated Schema Discovery:** Dynamically identifies URL and class label columns across varying dataset headers (e.g., `url`, `link`, `target`, `status`).
- **Label Inversion Normalization:** Reconciles divergent public benchmark conventions (e.g., PhiUSIIL where 1 denotes legitimate and 0 denotes phishing) into a unified standard: **0 = Legitimate (Benign)** and **1 = Phishing (Malicious)**.
- **Noise & Float Filtering:** Discards corrupted numeric strings, non-URL noise, and empty records.
- **Stratified Train-Val-Test Split:** Samples are split **80% Train** (142,527 URLs), **10% Validation** (20,361 URLs for early stopping), and **10% Test** (40,722 URLs for final performance scoring).
- **Class Imbalance Compensation:** To counter class imbalances, Lurelock dynamically computes the scale weight: `scale_pos_weight = N_negative / N_positive`, ensuring the loss function places equal emphasis on rare, evasive attack links.

3. The 1,018-Dimensional Feature Representation Space

In `src/feature_extractor.py`, Lurelock feeds the raw URL into a scikit-learn `FeatureUnion` that simultaneously extracts two complementary feature sets:

A. 18 Structural & Lexical Domain Indicators (F0 – F17)

Index	Feature Name	Computation	Phishing Detection Purpose
`F0, F1, F2`	URL, Host & Path Length	`len(url), len(host), len(path)`	Attackers pad URLs to push the real domain off-screen in mobile browsers.
`F3, F4`	Dot (`.`) Counts	`url.count('.'), host.count('.')`	Multiple dots denote nested subdomains impersonating trusted brands.
`F5, F6`	Hyphen (`-`) Counts	`url.count('-'), host.count('-')`	Hyphens are heavily used in spoofed names (e.g., `paypal-verification.com`).
`F7`	At-Symbol (`@`) Count	`url.count('@')`	In RFC 3986, characters before `@` are discarded as credentials, redirecting the browser.
`F8, F9, F10`	Delimiters (`?` , `=` , `/`)	Query and path delimiter counts	Measures path depth and evasion tokens designed to confuse web filters.
`F11`	Digit Count	`sum(c.isdigit() for c in url)`	Phishing and burner hosts use random strings with heavy numerical digit counts.
`F12`	Shannon Domain Entropy	`-∑ p(c) · log₂(p(c))`	Measures character randomness; high entropy flags algorithmically generated domains (DGAs).
`F13`	HTTPS Scheme Flag	`1.0 if https else 0.0`	Identifies unencrypted HTTP endpoints hosting sensitive login forms.
`F14`	Bare IPv4 Host Flag	`1.0 if host is IP else 0.0`	Bypasses DNS ownership; legitimate consumer services rarely use raw IPs.
`F15`	Subdomain Depth	`max(0, labels - 2)`	Identifies excessive nesting (e.g., `login.bank.com.attacker.com`).
`F16, F17`	Suspicious Keywords	Presence of 19 credential keywords	Flags urgency tokens (`login` , `verify` , `account` , `secure` , `update` , etc.).

B. 1,000 Character N-Gram TF-IDF Features (F18 – F1017)

Attackers use character substitutions (e.g., `rnicrosoft` , `paypa1`) that evade word tokenizers. Lurelock extracts sub-character n-grams using `TfidfVectorizer(analyzer='char', ngram_range=(3, 5), max_features=1000)` with sublinear term frequency:

$$w_{\text{tf-idf}}(t, d) = (1 + \log(tf_{t,d})) \times \log(1 + N / df_t) \quad \text{(Eq. 1)}$$

This captures brand syllables and evasion roots without maintaining giant dictionaries, operating comfortably within a memory footprint of **< 1.5 GB RAM**.

4. XGBoost Mathematical Optimization Formulation

Lurelock uses Extreme Gradient Boosting (XGBoost) with tree histogram aggregation (`tree_method='hist'`). At each boosting round t, the algorithm fits an additive regression tree $f_t(x)$ to minimize the second-order Taylor-approximated loss:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{N} [\, g_i \cdot f_t(x_i) + \tfrac{1}{2}\, h_i \cdot f_t^2(x_i) \,] + \gamma\, T + \tfrac{1}{2}\, \lambda \sum_{j=1}^{T} w_j^2 \quad \text{(Eq. 2)}$$

Where:

- $g_i = \partial\, \ell \,/\, \partial\, \hat{y}_i = p_i - y_i$ is the 1st-order gradient (the prediction error/residual).
- $h_i = \partial^2\, \ell \,/\, \partial\, \hat{y}_i^2 = p_i(1 - p_i)$ is the 2nd-order Hessian (local loss curvature).
- T is the number of terminal leaves, γ controls tree pruning, and λ is the L2 leaf regularization penalty.

Derivation of the Optimal Leaf Weight (w_j^*)

For a leaf j containing the set of samples $I_j = \{i \,/\, q(x_i) = j\}$, the objective with respect to leaf weight w_j is:

$$\mathrm{Obj}(w_j) = (\textstyle\sum_{i \in I_j} g_i)\, w_j + \tfrac{1}{2}\, (\sum_{i \in I_j} h_i + \lambda)\, w_j^2 \quad \text{(Eq. 3)}$$

Taking the first derivative with respect to w_j and setting it to zero yields the exact **optimal leaf weight formula**:

$$w_j^{*} = -\textstyle\sum_{i \in I_j} g_i \;/\; (\sum_{i \in I_j} h_i + \lambda) \quad \text{(Eq. 4)}$$

5. How Lurelock Calculates Risk Using the Weights

During real-time inference in the Chrome extension and local API server, calculating risk is deterministic and executes in **< 5 ms**:

Step 1: Decision Tree Traversal

The 1,018-dimensional feature vector x is passed through each of the 300 trained decision trees $k = 1, \ldots, 300$. In tree k, x traverses internal nodes comparing feature thresholds (e.g., `F11 (digits) < 5`, `F12 (entropy) < 3.8`). When it reaches terminal leaf $q_k(x)$, it retrieves that leaf's precomputed weight $w_{q_k(x)}$.

Step 2: Cumulative Margin Summation (Logit Accumulation)

The model sums the continuous weights from all 300 trees, scaled by the learning rate shrinkage parameter ($\eta = 0.08$):

$$z(x) = \sum_{k=1}^{300} \eta \cdot w_{q_k(x)} = 0.08 \times (w_{\text{tree1}} + w_{\text{tree2}} + \ldots + w_{\text{tree300}}) \quad \text{(Eq. 5)}$$

A negative margin $z(x) < 0$ indicates benign URL characteristics, while a strongly positive margin $z(x) > 0$ indicates accumulated phishing patterns.

Step 3: Logistic Sigmoid Probability Mapping

To convert the unbounded logit $z(x) \in (-\infty, +\infty)$ into a calibrated probability $p(x) \in [0, 1]$, Lurelock evaluates the logistic sigmoid function:

$$p(\text{phishing} \mid x) = \sigma(z(x)) = 1 \;/\; [\, 1 + e^{-z(x)} \,] = 1 \;/\; [\, 1 + \exp(-\textstyle\sum_{k=1}^{300} 0.08 \cdot w_{q_k(x)}) \,] \quad \text{(Eq. 6)}$$

Step 4: Discrete Threat Percentage & Safety Tier Classification

In `src/explainer.py`, the probability is mapped to an integer percentage score (0 – 100%):

$$\text{Risk Score} = \text{round}(\, p(\text{phishing} \mid x) \times 100 \,) \quad \text{(Eq. 7)}$$

Risk Score	Internal Level	User Interface Badge	System Classification & Guidance
0% – 29%	`safe`	✓ LIKELY LEGITIMATE	Normal structure matching legitimate domains. Safe to navigate.
30% – 59%	`warn`	! SUSPICIOUS PATTERNS	Multiple structural anomalies. Verify domain before submitting credentials.
60% – 100%	`bad`	✗ HIGH RISK PHISHING	High probability of credential theft or impersonation. Block / alert user.

High-Trust VIP Apex Domain Bypass: Certain legitimate platforms (such as `discord.com` channel links or `github.com` commits) contain deep numerical paths that could trigger digit thresholds. Lurelock includes a domain whitelist bypass in `src/domain_utils.py` and `server.py` that assigns **Risk = 0%** to verified corporate domains, preventing false positives.

6. Concrete Numerical Calculation Walkthrough

Consider an evaluation of the malicious link: `http://paypal-security-update.com/login.php`

1. **Feature Extraction:** `F0 (length) = 44`, `F5 (hyphens) = 2`, `F13 (HTTPS) = 0`, `F16 (keyword in host) = 1 (security)`, `F17 (keyword in path) = 1 (login)`, plus matching TF-IDF character n-grams (`'pay'`, `'sec'`, `'upd'`).
2. **Tree Traversal & Weights:** In Tree 0, the feature vector branches to leaf 8, returning weight $w = +0.1595$. In Tree 1, it reaches leaf 10, returning $w = +0.1471$. Across all 300 estimators, the trees output positive weights:

   $$z(x) \approx 0.08 \times (+131.75) = +10.54$$

3. **Sigmoid Calculation:**

   $$p(x) = 1 \,/\, (1 + e^{-10.54}) = 1 \,/\, (1 + 0.0000264) \approx 0.999974 \rightarrow \textbf{Risk Score} = \mathbf{100\%}$$

7. Production Benchmark Validation (Test Set: 40,722 URLs)

Evaluation Metric	Test Performance	Production Security Impact
ROC-AUC	**0.9996**	Exceptional discriminative separation between benign and phishing classes.
Accuracy	**99.87%**	Near-perfect classification across balanced test URLs.
Phishing Precision	**99.98%**	Ensures legitimate websites are almost never falsely blocked.
Phishing Recall	**99.62%**	Successfully flags 99.62% of evasive phishing attacks.
Phishing F1-Score	**0.9980**	Optimal balance of defensive coverage and user trust.